

Galv: Metadata Secretary for Battery Science

Brady Planden ^{1¶}, Matt Jaquiere ², Martin Robinson ², and David A. Howey ^{1,3}

¹ Department of Engineering Science, University of Oxford, Oxford, UK ² Research Software Engineering Group, Doctoral Training Centre, University of Oxford, Oxford, UK ³ The Faraday Institution, Harwell Campus, Didcot, UK ¶ Corresponding author

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Open Journals](#) 

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#))

Summary

The Galv project is a data and metadata management platform for battery science. Labs conducting battery research can use the platform to store and share (meta)data created from experimental electrochemical experiments. This data includes information about the electrochemical cell, the experiments, and the results. The platform provides a REST API for programmatic access to the data and metadata, in addition to a web interface for user interaction. The web interface provides additional functionality for importing and exporting data to widely supported data science languages, such as Python, Julia, and MatLab. This enables users to curate the experimental data within a single platform before completing advanced data analysis in their preferred language. The platform is designed to be extensible, allowing users to define their own metadata schemas and data types. This platform can be self-hosted or used as a cloud service enabling wider collaborations and sharing of experimental test data.

Statement of need

Galv is designed to address the need for a centralised data and metadata management platform for battery science. Battery research is a rapidly growing field, with many labs conducting experiments and generating data. This data is often stored in a variety of formats, making it difficult to share and compare results. By providing a centralised platform for storing and sharing data and metadata, Galv aims to make it easier for researchers to collaborate and build on each other's work. Furthermore, by defining a minimal structure for data, Galv can help researchers to ensure that their data is interoperable for meta-analysis and data science purposes.

Architecture

The Galv platform is composed of three key components, the backend, the frontend, and the harvester. The backend stores the corresponding (meta)data in parquet containers with a REST API that provides programmatic access for the frontend as well as third-party applications. The frontend is a web interface that allows users to interact, modify, and export the (meta)data stored in the platform. Finally, the harvester is a python package that can be used to automatically collect data and metadata from experiments and store it in the platform. The interaction of these three components is shown in [Figure 1](#) below.

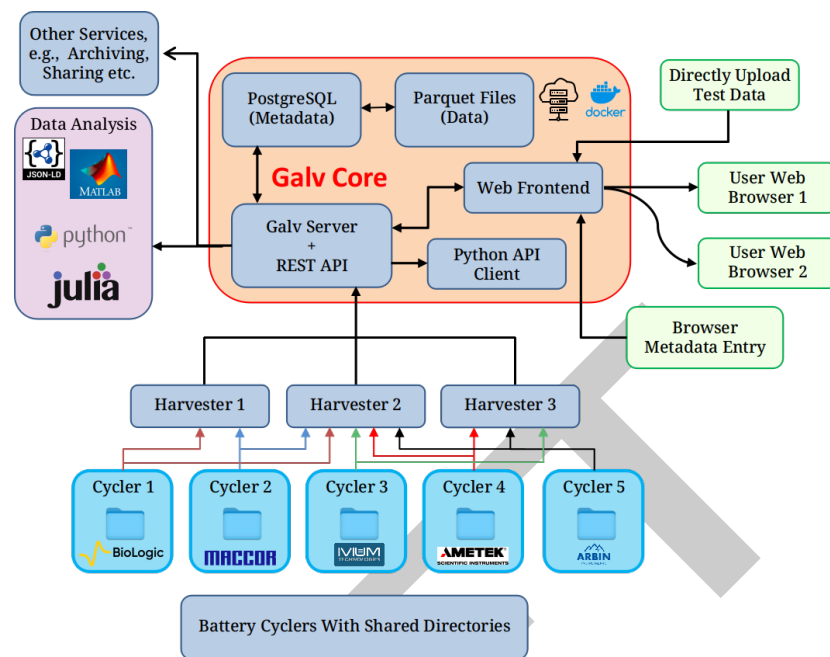


Figure 1: Galv's core architecture (backend) alongside data ingestion (harvesters), and the user web interface (frontend).

Galv is deployed as individual components with Docker images. These images can be built from the current state of development, or through images built during a Galv release and hosted on GitHub's container registry.

The backend server

The Galv backend is implemented with the Django framework (Foundation, 2024), with a corresponding REST API. This backend provide storage, authentication, and account management for the platform.

Items to discuss:

- Tweaks to the Django backend (superuser, example data)
- Parquet storage mechanism and performance over initial SQL database (i.e. splitting data and metadata for speed)
- Local storage and S3 Bucket

The frontend interface

The web-based frontend is implemented with the React framework and interfaces with the backend via the REST API. The frontend is the direct access for users' to the platform and is designed to provide the majority of the metadata curation. The frontend is structured to give end-users account management, split into individual user level, team-based, and laboratory. Each of these levels provides a structure tier for safe management of the underlying experimental data. The aim is to provide a structure that aligns with standard laboratory permissions and organisation.

Items to discuss:

- Metadata entry and organisation
- Time-series data rendering, application of CSV importing w/ header application (and storage for future events)
- Attachement of additional metadata in general object format to the underlying data (Image based datasheets, X-ray imaging, etc.)

63 The Harvester package

64 The harvester package is deployed through PyPI and provides the link between the experimental
65 data acquisition within the laboratory and the Galv platform hosted either locally or cloud-based.
66 Where possible, the harvesters parse the data files generated by the electrochemical cyclers
67 and provide the backend a parquet converted container for storage. Currently, the supported
68 proprietary data files include: Biologic, Maccor, and Ivium; however, data files stored in CSV
69 format are parsed with the user providing mapping of the headers to the exportable object
70 vectors in the frontend. Multiple harvesters can be setup on a single machine, with each
71 monitoring a specified directory. Each of these harvesters is provided an API key on creation,
72 which is generated from the specific backend instance and accessible from the frontend. As
73 each harvester is connected to a specific instance of a Galv backend, multiple Galv instances
74 can be used to monitor specific machines, which while not the main aim of the project, does
75 showcase Galv's flexibility as a data acquisition platform.

76 Items to discuss:

- 77 - Harvester options
- 78 - The ability to regex the directory
- 79 - Galvani repository reference

80 Key functionality

81 a. Data management

82 Galv provides a robust and flexible data management system designed to facilitate the orga-
83 nization and storage of experimental data related to battery research. It uses the Parquet
84 format to store data, which ensures fast and efficient data access. Additionally, PostgreSQL is
85 used for managing metadata, enabling quick retrieval and organization of experimental details.
86 The platform supports flexible and extensible metadata schemas that users can define based
87 on their specific research needs, allowing them to tag and retrieve data efficiently. Users can
88 upload, view, and download datasets seamlessly through the web interface or programmatically
89 via the REST API, ensuring accessibility for various workflows. Galv support data import and
90 export in different widely-used data science languages like Python, Julia, and MatLab, ensuring
91 smooth data transfer and processing across different platforms. Integrated tools support the
92 conversion of proprietary battery data formats (e.g., Biologic, Maccor, Ivium) into a unified
93 structure. This process significantly improves workflow efficiency by eliminating the need for
94 manual reformatting and enhances accuracy in downstream analysis by standardizing the input
95 data.

96 b. Access control

97 Galv implements a tiered access control system to ensure secure and organized management
98 of data. Access is managed at multiple levels. Individual users have private accounts for user-
99 specific data, while teams benefit from collaborative groups with shared permissions for specific
100 projects or experiments. Lab functions as higher-level entities for overarching management of
101 multiple teams and projects. Administrators can assign roles, manage permissions, and oversee
102 data access within their domain, ensuring compliance with institutional guidelines and fostering
103 collaboration while maintaining data security.

104 c. Validation schemas

105 Validation schemas serve as powerful tools for ensuring the integrity and consistency of data
106 within files and validating the metadata associated with other resources in the Galv ecosystem.
107 These validation schemas are defined using the JSON Schema specification, a widely adopted
108 standard for describing and validating JSON data structures. This flexible format enables
109 the creation of schemas capable of validating any JSON data, regardless of its complexity or

110 structure. By default, Galv applies a loose validation schema to all data, ensuring adherence to
111 the fundamental requirement of being valid JSON. Additionally, this default schema checks for
112 the presence of the minimal required columns: “time”, “potential difference”, and “current” –
113 essential columns for most battery cycling experiments and analyses. However, users have the
114 flexibility to define and apply more stringent validation schemas tailored to their specific needs.
115 These custom schemas can enforce additional constraints, such as data types, value ranges,
116 and complex relationships between different fields.

117 **d. Automatic unsupervised data harvesting**

118 Galv features an automatic unsupervised data harvesting system that collects data directly
119 from experimental setups without requiring manual intervention. Using the Harvester package,
120 the platform can automatically collect data files from specified directories on the machine
121 where the data is being generated. Harvesters monitor directories for new data files, process
122 them, and upload them to the Galv platform in Parquet format. The system is capable of
123 parsing proprietary data formats from various battery cyclers, such as Biologic, Maccor, and
124 Ivium. Additionally, the Harvesters can be set up to monitor multiple directories, allowing for
125 continuous, real-time data collection and storage.

126 **e. Design battery cyler experiment**

127 Galv allows users to design and manage battery cyler experiments by creating and defining
128 Cyler Tests. A Cyler Test encapsulates all necessary metadata associated with a specific
129 experiment, including the battery cell, equipment used, and experimental schedule. Users can
130 create a Cyler Test by selecting the appropriate resources, such as Cells, Equipment, and
131 Schedules, and associating them with the data gathered during the experiment. This system
132 enables researchers to track, organize, and analyze experimental data within the context of the
133 specific conditions under which it was obtained.

134 **f. Automatically parse cyler data**

135 Galv provides functionality to automatically parse and structure large cyler datasets. The
136 platform's parsing mechanism ensures that data from battery cyclers is formatted according
137 to Galv's standardized schema. The system renames columns, applies predefined mappings,
138 and organizes the data in a way that makes it easier to analyze and compare results. Large
139 data files are partitioned into smaller, more manageable chunks, allowing users to access
140 the data efficiently without having to deal with overwhelming file sizes. This feature is
141 particularly useful for researchers dealing with extensive datasets, ensuring that the data is
142 both accessible and ready for analysis. Initially, all harvested data files appear on the Files
143 page marked as 'GROWING'. As the Harvester detects file stability, the status transitions
144 to 'IMPORTING' and finally to 'IMPORTED' once fully processed. During this process,
145 Galv performs 'mapping' by renaming specific columns to align with standardized naming
146 conventions, such as ElapsedTime_s for time, Voltage_V for voltage, and Current_A for
147 current. Users can either rely on default mappings or define custom ones tailored to their
148 needs. For files with an applicable mapping, Galv automatically applies it, ensuring consistent
149 data structure. On the Files page, imported files are displayed alongside a preview image of
150 the data. Users can click on a file to access detailed views or download the dataset.

151 **Acknowledgements**

152 We would like to acknowledge all contributors to Galv (link), and the funding bodies that
153 supported this work: IntelLiGent, DigiBatt, Energy Superhub?, Digest.

154 **References**

- 155 Foundation, D. S. (2024). *Django: The web framework for perfectionists with deadlines*
156 (Version 5.1.3). Django Software Foundation. <https://github.com/django/django>

DRAFT